

Catalogizer

高级多协议媒体库管理——自动检测、编目与丰富你拥有的一切

概要

Catalogizer 是一款可自托管的媒体库管理系统，能自动检测、分类并整理跨 SMB、FTP、NFS、WebDAV 及本地文件系统的媒体内容，具备实时监控、加密存储、外部元数据丰富及现代化 React 界面，后端由高性能 Go API 驱动。

简介

一款生产级多协议媒体库管理工具。Go/Gin REST API 可检测 SMB/FTP/NFS/WebDAV/本地源中的 50 余种媒体类型，从 TMDB/IMDB/MusicBrainz/Steam 等平台丰富元数据，并通过加密 SQLCipher 数据库提供实时 React Web 应用。

详细说明

多数媒体管理工具要求你先做出妥协：将所有内容整合到同一磁盘、同一格式、同一类型，然后才提供帮助。Catalogizer 则从相反的前提出发——你的媒体库早已分布在各处，散落于永远无法统一的 NAS 共享和协议中——并直接与之对接。它支持存储系统已采用的协议——SMB/CIFS、FTP/FTPS、NFS、WebDAV 及本地文件系统——并通过统一的客户端抽象层将它们整合，使 Windows 共享、FTP 归档和 WebDAV 挂载在上层应用看来完全一致，可随意混合、替换或移除，无需修改应用代码。Go 后端（Gin REST API）持续监控这些数据源，在文件出现时即时检测并分类 50 余种媒体类型（电影、电视剧、音乐、游戏、软件、纪录片等），并通过 TMDB、IMDB、TVDB、MusicBrainz、Spotify、Steam 等外部平台丰富每个条目，将原始文件名转化为包含封面、演职员及元数据的完整编目条目。结果通过 WebSocket 实时推送至 TypeScript React 前端，媒体库在摄入过程中即时更新，无需手动刷新，所有元数据均存储于加密的 SQLCipher 数据库中，并通过基于 JWT 的角色权限认证进行访问控制。

多数编目工具在共享源断开时便悄然失效，而 Catalogizer 则专为应对此类故障而设计。临时 SMB 连接中断会触发指数退避重连、断路器机制（避免对失效主机持续请求）、持续健康监测，以及离线元数据缓存，确保在最后

已知良好状态下继续响应用户请求——区别在于，前者可能因“一个 NAS 重启导致整个应用崩溃”，而后者则是“单个数据源降级，其余功能正常运行”。除了编目功能，它还可作为媒体库的运维工具：提供增长趋势与质量/版本跟踪分析、专业 PDF 报告生成、PDF 至图片/文本/HTML 的转换服务、收藏夹导出/导入 (JSON/CSV)，以及与 S3、Google 云存储或本地文件夹的同步功能。它并非一个庞大的单体应用，而是由 21 个可复用的 `digital.vasic.* Go` 子模块及 TypeScript 客户端包精心组装而成，每个模块均经过独立测试与版本控制，相同的经过实战验证的认证、文件系统、流媒体及可观测性组件不仅驱动 Catalogizer，还支撑着更广泛的产品家族。质量保证并非自说自话：Challenges 框架与 HelixQA 确保每项宣称的功能均经过反虚假验证，并提供证据支撑。

内容

我们为何构建它

现有的媒体管理工具均假设单一存储后端与单一媒体类型。而真实的收藏分布于多个 NAS 共享与协议中，一旦某个共享失效便会导致系统崩溃，且涵盖电影、音乐、游戏及软件等多种类型。Catalogizer 的诞生，正是为了平等对待所有协议，在网络存储不稳定时依然稳定运行，并为所有内容提供一个权威、丰富且加密的统一目录。

它为何是颠覆性的

它将通常需要一整套独立工具才能实现的功能整合为一个可自托管、加密的完整方案：协议无关的摄取机制，平等对待每个存储后端；弹性设计确保存储中断时目录依然可用，而非随之崩溃；多源丰富化处理将原始文件转化为可浏览、带属性的资源库。模块化架构的回报是复利效应：文件系统客户端的加固修复或新增的提供商插件只需部署一次，便能惠及所有使用者，使 Catalogizer 随着周边生态的完善而持续进化。简而言之，它不再是一个简单的媒体索引，而是一个真正的媒体系统——一个你拥有主权、能抵御基础设施波动、且内部机制经实践验证而非仅凭承诺的系统。

创新之处

- 统一多协议文件系统客户端 (SMB/FTP/NFS/WebDAV/本地存储)，通过单一接口调用。
- 离线缓存 + 熔断机制，确保存储中断时目录依然可用。
- 完全模块化提取，形成 21 个可复用的 `digital.vasic.* Go` 子模块及 TS 客户端模块。
- 静态加密目录 (SQLCipher)，通过 WebSocket 实时同步至 UI。
- 基于挑战框架与 HelixQA 集成的证据驱动质量保障。

挑战与解决方案

- 不稳定的网络存储：通过指数退避、熔断机制、健康检查及带驱逐策略的离线缓存解决，当源不可达时提供缓存元数据服务。
- 协议异构性：通过将所有协议抽象为统一的`digital.vasic.filesystem`客户端解决，上层逻辑无需感知具体协议。
- 数据安全：通过SQLCipher静态加密、JWT/RBAC权限控制及请求清洗中间件解决。
- 大规模可维护性：通过将通用逻辑提取为独立测试的子模块而非单体架构解决。

技术栈（选择原因与实现方式）

- **Go + Gin** —— 高性能REST API核心（`catalog-api`），针对持续监控工作负载的并发与吞吐优化。
- **TypeScript + React + Tailwind (Vite)** —— 响应式`catalog-web` UI，支持实时更新。
- **WebSockets** —— 后端中心与UI之间的实时数据同步。
- **SQLCipher（加密SQLite）** —— 静态加密元数据存储，通过`digital.vasic.database`支持双SQLite/PostgreSQL。
- **SMB/FTP/NFS/WebDAV客户端** —— 通过`digital.vasic.filesystem`实现多协议摄取。
- **外部元数据API（TMDB、IMDB、TVDB、MusicBrainz、Spotify、Steam）** —— 丰富化提供商插件。
- **Prometheus + OpenTelemetry** —— 通过`digital.vasic.observability`实现指标与追踪。
- **Docker / 构建容器** —— 可复现构建（Tauri/Rust通过`catalogizer-builder`路由）。
- **Redis** —— 通过`digital.vasic.cache / ratelimiter`实现缓存与限流。
- **S3 / Google云存储** —— 云同步与检查点存储。